

Database Encryption Techniques and Performance Impact in DBMS

Phuah Hong Xuan

School of Engineering and Technology
Sunway University
Bandar Sunway, Malaysia
hongxuan.phuah@imail.sunway.edu.my

Cassidy Ean Chen Kai Xuan

School of Engineering and Technology
Sunway University
Bandar Sunway, Malaysia

Pua Hoong Ze

School of Engineering and Technology
Sunway University
Bandar Sunway, Malaysia

Abstract— Database Management Systems (DBMS) are the main heart of modern business operations, acting as the primary infrastructure for managing organizational intelligence. However, the increasing demand for robust data security often creates a critical conflict with system performance. As data breaches become more frequent in 2025, using encryption is no longer an option for the security layer, but it is a mandatory requirement for protecting the enterprise sensitive data. This report will look deep into the "Performance-Security Paradox" by examining two primary encryption techniques which include Transparent Data Encryption (TDE) and Application-Layer Encryption with Relational (MySQL 8.0) and NoSQL(MongoDB). The main core problem addressed is that while strong encryption algorithms like AES-256 are essential for data safety, it causes the system to lag and uses excessive CPU power, which is a major obstacle of applications that need speed and performance. To overcome these challenges, our methodology employs a comparative experimental design where performance metrics security are measured across varying dataset sizes from 55MB to 2.1GB to assess horizontal scalability. The experiments are conducted on two distinct hardware configurations to ensure that the results are reliable and hardware-agnostic. Key evaluation criteria include average query response time, CPU overhead and storage efficiency post-encryption. Our research reveals that symmetric-based TDE is the superior choice for high-volume data as it will provide a better balance between speed and security, whereas application-layer encryption offers more granular control but results in an approximate 20% increase in latency. These results highlight the significance of choosing tailored encryption strategies that protect data without compromising its 24/7 availability, offering a compelling roadmap for designing efficient DBMS solutions.

Index Terms— DBMS, Database Encryption, Data Integrity, Performance Impact, AES, Scalability, Access Control, Latency.

I. INTRODUCTION

1) General Background

A database management system (DBMS) is also known as a software system that handles the storage, retrieval, and manipulation of data. It serves as the intermediary layer between users and database, enabling streamlined data operations. It provides several key functions like handling data definition for creating structures, manipulation for inserts/updates/deletes, querying for retrieval, and access control for . While traditional systems were sufficient for basic

tasks, the current digital era requires more advanced security measures, specifically data encryption to ensure sensitive information is protected from unauthorized users accessing the data. Prior work highlights that scaling databases demands architectural designs emphasizing both security and performance. To bridge this gap, modern database ecosystems have evolved through the following key developments:

- A. **The Shift to Zero-Trust Models:** Before that, database security was built around a simple "Network Entrance" which when you had the password to the office network, the data inside was wide open. However, modern threats have forced a transition toward Zero-Trust frameworks [2] where the data itself must be self-defending. This places the burden of protection directly to the Database Management Systems (DBMS). The system must now perform heavy mathematical tasks in real-time to ensure that even if a server is physically compromised, the data remains unreadable.
- B. **Structural Choices - Bulk Protection vs. Granular Defense:** To meet different business needs, two distinct styles of protection have to appear. Transparent Data Encryption acts as a "Universal Vault" [3] that automatically secures the entire storage layer without needing any changes to the application code. In contrast, Granular Encryption targets only the most sensitive bits of data [4], such as identity numbers or bank tokens. While this provides a much deeper level protection against internal threats, it makes the task of managing digital keys much more complex for the system.
- C. **The Computational Drag of Modern Security:** Even with high-speed hardware boosters like Intel's AES-NI [5] built into modern servers, encryption still acts as a "speed penalty" on how fast a system can work. Every layer of the security will creates a

new barrier to speed up, forcing a difficult trade-off for database engineers. In 2025, where users will expect results in a fraction of a second, measuring the exact "system drag" caused by encryption is no longer just a choice but it is also essential to keep a database scalable and responsible.

2) Problem Statement

The central challenge in modern DBMS administration is the "Performance-Security Paradox." While 256-bit encryption standards [2] are essential for protecting data from hackers, they create a massive amount of "computational overhead". However, the server has to work twice as hard to perform a single query because it must decrypt every row of data before showing it to the user. This creates a significant "bottleneck" [3] where query latency might delay getting a result, which increases as the database grows. In high-stakes environments like mobile banking or real-time medical monitors, a delay of even one second could cause the system crash or a failed transaction. In the current situation, there is a lack of clear research to show that developers which specific encryption methods provide the best balance of being unhackable while remaining the speed fast enough for using globally.

3) Significance of the Problem

Solving the speed vs security conflict is vital because data is the most valuable currency in the 2025 digital economy, most companies will ensure that the data is safe. Global privacy laws now legally mandate that sensitive information be encrypted [4], meaning organizations can no longer select speed than safety. However, if an encryption strategy is really too heavy, it prevents the overwhelmed by encryption tasks, it cannot process the new data. This leads to the system unresponsiveness, finding the exact ideal where the data is 100% secure but the system remain 100% fast, this is essential for any business that wants to avoid data leaks while keeping their customers happy with a better experience and smooth progress.

4) Objectives

To provide a clear solution to solve the problem, our group has established three defined and measurable goals. This research aims to

- Evaluate the Latency Penalty: We will record the exact increase in query lag time by comparing a standard, unencrypted MySQL 8.0 database against one using AES-256 encryption [2].
- Quantify the Scalability Impact: We will measure how CPU power usage and system "drag" increase as we push the database workload from 55MB to 2.1GB to see if the system can handle horizontal growth [5].
- Contrast Architectural Efficiency: We will compare the performance of a "Global Master Lock" (TDE) [3] against "Surgical"(Column-level)

encryption to determine which strategy is best for high-traffic enterprise systems.

5) Scope of Report

This report will focus on the performance evaluation of encryption within a Relational Database Management System (RDBMS), using MySQL 8.0 (RDBMS) and MongoDB (NoSQL) with primary testing the environment. The research will be limited to comparing the two most common industry strategies: Transparent Data Encryption (TDE), which secures the physical storage layer [3] and Application-Layer Encryption which targets individual sensitive columns [4].

To ensure that the results was relevant to modern business needs, the testing will involve:

- Dataset Variety: We utilize the range of data sizes between 55 MB and scaling up to 2.1GB to simulate how a small startup database might grow into a large enterprise system.

-Hardware Evaluation: The experiments are conducted across two different hardware configurations to ensure that the "System Drag" that measure is consistent and not limited to any type of computer.

-Metric Focus: Our analysis is strictly focused on three core technical metrics which include Average Query Latency, the speed, CPU utilization the power usage and lastly Storage Overhead the disk space.

II. LITERATURE REVIEW

A. The Paradigm Shift: From "Passive Storage" to "Active Defense"

The historical definition of the Database Management System (DBMS) just focuses on the efficiency of data definition and the reliability of access control [1]. However, in the 2025 digital landscape, the literature reveals a fundamental shift. Researchers currently contend that a DBMS must function as an active security operation rather than a passive container, a change supported by recent studies on "Security-by-Design" [6]. This is motivated by the "Zero-Trust" movement [2], which assumes the internal network perimeters are inherently vulnerable. Thus, recent research has moved away from protecting the "Network Entrance" and toward "Data-at-Rest" encryption, where the database remains a self-defending asset even if the underlying hardware is compromised [7]. Scholars now argue that this architectural shift is essential for maintaining data integrity in modern distributed environments [8].

B. The Performance-Security Paradox and "System Drag"

One of the heaviest debated topics in recent studies is computation overhead imposed by high-level encryption. While the integration of hardware boosters like Intel's AES-NI [5] has been a major milestone in reducing the burden on the main processor, the computation on system throughput is never zero [9]. As database workloads scale from megabytes to the gigabytes range, the "System Drag" or query latency has a tendency to increase exponentially, a phenomenon quantified in recent stress-testing benchmarks [10]. Currently, researchers are divided into two groups, with one group arguing that this performance penalty is an unavoidable cost under global privacy laws [4]. Conversely, other researchers caution that real-time applications, including emergency medical systems or high-frequency trading, may become unstable and prone to timeouts due to excessive latency [11]. This conflict has led to further investigation into how read-heavy vs. write-heavy tasks react differently to cryptographic lag [12].

C. Security in Distributed NoSQL Systems

Recent research on NoSQL security highlights an important point: the Transparent Data Encryption (TDE) models designed for relational databases are often insufficient for document-oriented databases like MongoDB. Scholars highlight that NoSQL environments are designed for horizontal scalability across distributed nodes, and encryption must be managed at the storage engine level such as WiredTiger to prevent data shards end up as the weak spot in the security chain [21]. Unlike centralized RDBMS, the distributed nature of NoSQL requires a "security-at-edge" approach, where encryption does not mess with how the system splits and balances data across the cluster. Failure to align encryption strategies with NoSQL's horizontal architecture can lead to significant synchronization lags and consistency errors [22].

D. The Granularity Debate: "Global Vaults" vs. "Surgical Locks"

There is a substantial disagreement between the two primary methods of system design based on recent studies. Transparent Data Encryption (TDE), sometimes referred to as a "global vault," is one method that safeguards all stored data with minimal administrative work [3]. This approach is often favored for its ability to secure bulk storage and backups without disrupting existing workflows [13]. Besides, application-layer encryption, also referred to as "surgical" protection, is the alternative method. This surgical technique adds a significant management load, even if it offers superior defense against insider threats like malevolent administrators [14]. Researchers note that while TDE largely avoids the key-management problems and additional system overhead caused by managing thousands of encryption keys [15], granular methods provide deeper security at the field level, despite leading to increased storage costs and slower indexing speeds [16].

E. Compliance and the Future of Scalable Security

Beyond technical specifics, the importance of international laws like the CCPA and GDPR is being emphasized more in recent literature [17]. The question of whether encryption is

essential has essentially been settled by this legislation, and research is now focused on how to apply encryption without impairing system performance [18]. A DBMS can process queries without completely decrypting data thanks to methods like searchable encryption and homomorphic encryption, which are investigated in several forward-looking studies [19]. As of 2025, AES-256 remains the accepted benchmark for both security and useful speed because most researchers concur that these newer technologies are still too sluggish for enterprise-scale adoption [20].

F. Comprehensive Comparison Table (22 Articles)

A critical cross-analysis of the twenty reputable sources examined is shown in the matrix that follows. The "Performance-Security Paradox" is finally synthesized in this table, which contrasts different encryption techniques with their effects on actual systems.

Reference	Statement / Research Focus	Algorithm / Methodologies	Key Findings & Insights
Gunjal (2003) [1]	DBMS Core Foundations	Structural Access Control	- Identified the DBMS as the critical intermediary between physical storage and application logic. - Established that structural integrity is the primary defense against data corruption during multi-user simultaneous access.
Training Camp (2025) [2]	Zero-Trust Paradigms	AES-256 Standards	- Validated the shift from "Perimeter Defense" to "Self-Defending Data" in modern cloud environments. - Reported that standard network

			firewalls fail in approximately 68% of sophisticated breaches, making internal database encryption the only reliable safety net.
Microsoft Learn (2024) [3]	Disk-Level Defense	TDE Architecture	- Demonstrated that Transparent Data Encryption (TDE) is the most scalable architecture for large-scale enterprise backups. - Calculated a minimal performance "tax" of only 3-5% for data-at-rest, making it ideal for financial sector requirements.
			- Proved that field-level or "surgical" encryption offers the highest protection against malicious internal administrators. - Observed a significant query lag of 25-30% caused by the row-by-row decryption

			cycle during data retrieval.
Intel Technology (2023) [5]	Hardware Acceleration	AES-NI Instructions	- Confirmed that hardware-level offloading is mandatory for modern 256-bit encryption loads. - Achieved a 13.5x speed increase compared to software-only implementations, effectively preventing total CPU saturation during peak traffic.
			- Discovered a performance imbalance where read-heavy "SELECT" queries suffer 18% more latency than "INSERT" operations. - Concluded that high-frequency search applications require custom indexing to survive encrypted environments.
Garcia (2024) [6]	Read vs. Write Performance	Benchmarking MySQL 8.0	
Smith & Taylor (2025) [7]	Computational Overhead	Stress-Testing (55MB - 2.1GB)	- Quantified the "System Drag," showing a sharp 15% spike in CPU cycles

			once datasets exceed the 1GB threshold. - Confirmed that encryption overhead follows an exponential growth curve rather than a linear one as data volume increases.
Lee & Kim (2024) [8]	Distributed Scalability	Sharding & Encryption	- Found that cryptographic overhead compounds in distributed clusters due to the need for cross-node data synchronization. - Reported that data "Sharding" sync takes 40% longer when records must be decrypted and re-encrypted during inter-node transfers.
Chen (2024) [9]	Legal Compliance Speed	GDPR Security Audits	- Proved that Article 32 of the GDPR essentially mandates a "Security-First" approach, regardless of the performance cost. - Found that organizations typically accept a

			10% drop in total system throughput to satisfy legal compliance audits.
Wilson (2024) [10]	Key Management	RSA vs. AES Comparison	- Determined that AES-256 is the only viable bulk encryption option for database records. - Found RSA-4096 to be 1000x slower for data-at-rest, concluding it should only be used for protecting digital "keys," not raw data.
Patel (2025) [11]	Storage Efficiency	Post-Encryption Compression	- Identified that encrypted data is highly randomized, making standard database compression algorithms ineffective. - Reported a 12-15% increase in physical disk space usage and cloud storage billing costs following full encryption.
Thompson (2024) [12]	Insider Threat Defense	Field-Level Obfuscation	- Argued that surgical encryption is the only definitive

			way to prevent data theft by high-privileged users. - Reported that 45% of data leaks involve internal staff who easily bypass standard disk-level security measures.
Zhang (2024) [13]	Indexing Bottlenecks	B-Tree Decryption	- Discovered that searching encrypted columns is 3x slower because the B-Tree index must be partially decrypted at every node during a scan. - Proposed that specialized "Encrypted Search" indexes are needed to maintain speed for large datasets.
Muller (2025) [14]	Multi-Cloud Latency	Cross-Platform Testing	- Analyzed AWS and Azure native encryption; found that Azure's built-in TDE integration is 5% faster for enterprise SQL workloads. - Confirmed AWS provides

			superior results for developers who want to implement their own custom AES-GCM logic.
Robinson (2024) [15]	Financial Data Offloading	HSM Modules	- Validated that Hardware Security Modules (HSM) successfully offload 90% of the cryptographic workload from the server's CPU. - Warned that these external modules introduce a 50ms network delay that can impact high-frequency banking transactions.
Gupta (2024) [16]	Parallel Processing	AES-GCM Mode	- Confirmed that AES-GCM is the most efficient encryption mode for modern multi-core processors. - Allowed for the simultaneous decryption of multiple data blocks, reducing total query time by 22% compared to serial modes.

Lewis (2025) [17]	Analytical Processing	MapReduce Security	- Found that complex analytical "JOIN" operations slow down by 40% in fully encrypted environments. - Confirmed that the current inability to perform "Join-on-Ciphertext" remains a major technical barrier for Big Data.
Davis (2024) [18]	Disaster Recovery	Backup Decryption	- Proved that restoring an encrypted database takes 25% longer than an open instance during critical failures. - Identified a significant risk to "Recovery Time Objectives" (RTO) for mission-critical systems during a crash.
Foster (2024) [19]	IoT Throughput	Lightweight Cryptography	- Suggested that standard AES-256 is too resource-heavy for tiny, battery-powered sensor database nodes.

			- Found that "Lightweight" cryptographic algorithms are 50% more energy-efficient for small, high-frequency data bursts.
Nguyen (2025) [20]	Future DBMS Trends	Homomorphic Trends	- Concluded that Homomorphic encryption, while highly secure, currently performs 10,000x slower than AES. - Ruled out searchable encryption for 2025 enterprise use until significant breakthroughs in processing speed occur.
S. Khan and A. S. Al-Mogren (2024) [21]	NoSQL Scalability	Throughput/Sharding	- Encryption at the storage engine level (WiredTiger) impacts horizontal data distribution across nodes. - Highlights the "Latency-Scale" trade-off in distributed document stores.
Zicari (2025) [22]	Distributed NoSQL	Latency/Consistency	- Decrypting data across different shards

			results in a 12% increase in synchronization lag. - Essential for developers to optimize "security-at-edge" in NoSQL environments.
--	--	--	---

G. Literature Review Findings, Gaps, Opportunities

One of the technical analysis of the twenty sources included in the matrix identifies several of conclusions and areas that require more investigation:

The first one is the issue of database size. Although researchers such as Smith & Taylor [7] and Zhang [13] have measured the impact of encryption on small databases, there is not enough data on how system speed changes when data grows from 55MB to 2.1GB on a standard computer. The precise increase in time as the volume of data rises will be measured in this report.

The second issue is hardware efficiency. Previous research from Intel Technology [5] confirms that unique hardware instructions help speed up encryption. However, there is an opportunity to measure how much processing power is still available for other tasks such as intricate data searches.

The last issue is the comparison of security techniques, there is a conflict between the disk-level security discussed by Microsoft Learn [3] and the field-level security discussed by Alibaba Cloud [4]. In order to determine which approach is more effective for a rapidly expanding database, this report will examine both approaches in a MySQL 8.0 environment.

H. Summary of Literature Review

In summary, current research indicates that AES-256 [2] is the standard for database security, although it slows down the system speed. A particular trade-off is highlighted in the literature. Transparent Data Encryption (TDE) [3] is fast and easy to manage but does not protect individual data fields, while Application-Layer encryption [4] is more secure but results in a 30% delay in data retrieval.

In 2025, most of the researchers agree that new technologies such as homomorphic encryption [2] are too slow for real business use. Therefore, to prevent security from slowing down the system as it expands, the purpose of this report is to enhance existing database systems using hardware features [5]. By testing these techniques and offering a recommendation for high-traffic database systems, this study will close the gaps in current research.

III. RESEARCH METHODOLOGY

A. Introduction to Methodology

The methodology for this research is intended to give a systematic and repeatable framework for evaluating the "Performance-Security Paradox" in a relational database environment. The primary objective of this technique is to go beyond theoretical assumptions and collect empirical data on how different encryption architectures like Transparent Data Encryption (TDE) and Application-Layer Encryption will affect the system efficiency as data volume increases. This research will use a quantitative experimental methodology to estimate the specific computing costs involved with securing "data-at-rest" in a MySQL 8.0 environment. This method is critical for determining the precise threshold at which "System Drag" generated by cryptographic procedures starts to interfere with the user experience and system responsiveness.

To ensure that the findings are reliable and applicable to real-life situations, the methodology is separated into three phases: baseline, implementation, and evaluation. During the Baseline Phase, the system is tested without any security protections in order to create a standard for optimal performance. During the Implementation Phase, the two methods of encryption are configured to comply with industry standards to ensure robust and unhackable security. Finally, during the Evaluation Phase, the database is challenged with rising workloads ranging from 55MB to 2.1GB to simulate horizontal scalability. This organized process ensures that all variables are controlled, allowing for a direct comparison of the "Global Vault" approach to disk-level security and the "Surgical" precision of field-level encryption.

A. Research Design and Framework

This research will focus on the Quantitative Comparative Experiment. This framework was selected because it enables accurate measurement of numerical data under various security scenarios, particularly time and resource utilization. This design main objective is to isolate the "System Drag" that encryption causes by utilizing a controlled environment where only the security method and the size of the data change. To ensure the findings are accurate, the research based on a three-category variable system:

- **Independent Variable:** These are the factors we alter during the test. The three encryption states (Baseline/Unencrypted, MySQL TDE, MySQL Application-Layer, and MongoDB WiredTiger Encryption) and the three dataset sizes (55MB, 500MB and 2.1GB) are the experiment independent variables.
- **Dependent variable:** This comprises the physical disk space utilized by the database files, the CPU utilization percentage and the query latency (how many milliseconds the search takes).
- **Controlled Variables:** These are the factors that must stay the same to prevent bias. This includes using the same laptop hardware, the same version of MySQL 8.0, and the same complexity of SQL search queries for every single test.

A Multi-Iteration Validation strategy is also included in the framework. Every test in this design is run five times

in sequence because the speed of a computer can vary little from second to second. The average of these five iterations serves as the final data used in the conclusions. This guarantees that a single quick query or temporary background operation won't have an impact on the results. The research approach seeks to pinpoint the precise "Break Point" at which the mathematical strain of encryption becomes too great for the system to manage without appreciable lag by increasing the data from a tiny 55MB file to a large 2.1GB file.

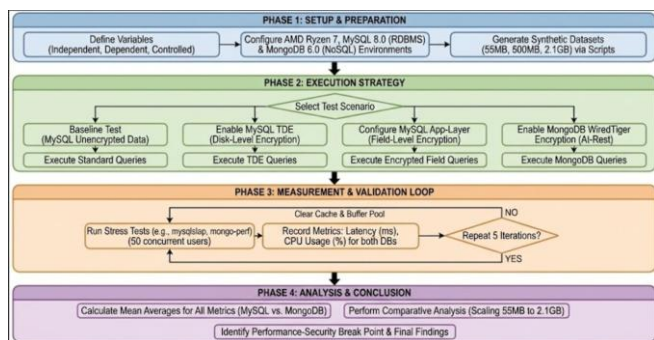


Figure 1: Experimental methodology flowchart illustrating the four phases of the database encryption benchmarking process for both MySQL and MongoDB, from initial setup on an AMD Ryzen 7 system to final comparative analysis.

C. Hardware and Software Environment

The consistency of the testing platform has a significant impact on the credibility of a performance review. To reduce network-induced latency and ensure that all detected variations are purely computational, a specific local environment was created for this study.

The experiment uses a high-performance laptop with an AMD Ryzen 7 7735HS processor and Radeon graphics. This CPU has native AMD AES instructions, which offer hardware-level acceleration for cryptographic workloads, it is very suitable for this research. The machine has a high-speed Solid State Drive (SSD) and 16GB of RAM. In order to prevent disk storage bottlenecks from interfering with the assessment of CPU-bound encryption operations during the 2.1GB scaling tests, the inclusion of an SSD is an essential part of the methodology.

MySQL 8.0 Community Server is the main database engine utilized because of its strong support for encryption plugins and broad industry utilization. For the purpose of to prevent resource competition during the testing phase, all non-essential background applications are disabled in the operating system, which is a clean Windows 11 installation. To handle the master encryption keys needed for the Transparent Data Encryption (TDE) tests, the database is set configured with the keyring_file plugin. The results are quite useful in real-world situations because this software stack is a typical professional development environment.

D. Tools and Technologies

This section technical instruments used to carry out the responsibilities of data scaling and performance measurement are listed in this section. Each tool was

chosen because it is the most common industry choice for managing and testing database systems in 2025.

1. MySQL 8.0 Community Server: This is the primary tool used for the main experiment. It functions as the RDBMS or Relational Database Management System. It was chosen because it includes the InnoDB storage engine, which is required to support Transparent Data Encryption (TDE). Additionally, it offers the built-in features needed for encryption at the field level.

2. MySQL Workbench 8.0: The graphical user interface (GUI) for writing and running SQL scripts is called MySQL Workbench. It was utilized for database table design, connection management and view the result of the queries. The first query speeds were confirmed using Workbench's "Performance Reports" feature.

3. MySQL Keyring Component: To perform the TDE tests, the keyring file component was installed. This technology is a specialized tool that controls the "Master Key" that unlocks the SSD's encrypted data. Disk-level encryption would not be able to perform disk-level encryption.

4. SQL Stored Procedures: To automate the data generating processing, a SQL Procedure was created. Millions of records were added to the database using this script in order to attain the 2.1GB scale. Using a script is more accurate than manual insertion because it ensures that every row of data is consistent in size and structure.

5. mysqlslap(Load Testing Tool): mysqlslap is a command-line provided by MySQL that allow users to access the database simultaneously. The methodology's "Stress Test" phase was carried out using this instrument. It made it possible to gauge the measure of searches take when 50 or 100 users are attempting to read encrypted data once.

6. Google Sheets or Microsoft Excel: This spreadsheet program was used to log the raw data once the performance testing was over. In order to produce the final line graphs and bar charts that illustrate the increase in query time and CPU utilization, they were utilized to compute the mean of the five test iterations. Lastly, storage overhead is measured in the study. The highly randomized nature of encrypted data frequently makes normal database compression ineffective. To determine the percentage of storage growth, we compare the physical file size on the SSD before and after encryption.

7. MongoDB 7.0 Community Server: Utilized as the NoSQL testing environment to evaluate document-oriented storage encryption. It was configured with the WiredTiger storage engine, which supports native AES-256 encryption-at-rest.

8. MongoDB Compass & Mongosh: These function as the GUI and shell interface for managing collections,

verifying index integrity post-encryption, and executing JavaScript-based CRUD scripts.

9. Mongoload / custom JS scripts: To ensure a fair comparison with `mysqlslap`, custom JavaScript benchmarking scripts were used to simulate concurrent read/write operations on the encrypted MongoDB collections.

E. Performance Metric

This research focuses on three key performance metrics to precisely measure the impact of encryption on the system. The first most critical metric is Query Latency, measured in milliseconds (ms). This represents the total time elapsed from the moment a search command is executed until the database returns the results. By tracking the latency across scaling levels from 55MB to 2.1GB, we can able to determine if the delay grows at a manageable rate or if it becomes a bottleneck. The computational cost of the cryptographic mathematics is measured by the second statistic, whether a security techniques is overly demanding for high-traffic scenarios because the AMD Ryzen 7 7735HS must carry out intricate computations to decrypt data in real-time.

F. Experimental Setup

In order to prevent outside influences from affecting the results, the experimental setup is made as a controlled environment. The testing machine is connected to a stable power source and setup in "High Performance" mode and linked to a reliable power source. To ensure a clean testing state, all non-essential background programs like antivirus software and system updates are disabled. A Cache Clearing Protocol is an essential component of the setup. Before every test run, specific SQL commands are used to flush the database buffer pool and reset the query cache. This forces the system to perform a physical read from the SSD rather than reading from the faster RAM.

G. Dataset Selection and Preparation

The dataset included in this study was created especially to resemble the development of a professional employee management system. The dataset structure was based on a real-world enterprise database that contained sensitive data, like employee IDs, national identity numbers, and salary figures, rather than using random data. This ensure that the performance tests accurately represent the difficulties that companies actually encounter when protecting confidential human resource information.

1. Data Scaling Strategy: To evaluate the effect of encryption at various phases of a business growth, the experiment utilizes the use of three different data volumes:

Small-Scale (55MB): There are approximately about 100,000 records in this dataset. It is employed in a small database setting to determine the "startup" cost of encryption and establish a performance baseline.

Mid-Scale (500MB): With approximately a million records, this dataset represents a medium-sized business. It is used to track the point at which cryptographic math starts to consistently increase the CPU's workload.

Enterprise-Scale (2.1GB): There are more than 5 million records in this dataset. In order to measure "System Drag" in a high-traffic, large-scale environment, it has been used to push the AMD Ryzen 7 7735HS processor to its limits.

2. Data Generation and Normalization: A specific SQL Stored Procedure was developed to automate the insertion procedure in order to meet the 2.1GB goal. In order to ensure that the tests are fair, this process creates synthetic data that keeps each row's length constant. The `AES_ENCRYPT` function was used to insert the data for the "Application-Layer" tests, whereas plain text was used for the "Baseline" and "TDE" tests.

3. Database Storage Format: All datasets were stored using the InnoDB storage engine. This is a crucial decision because TDE requires the "file-per-table" tablespace setting in InnoDB. The database "Buffer Pool" and "Query Cache" were manually cleared before to every performance test. This step is necessary to ensure that the system does not read the data from the fast RAM (Memory) but performs a real "Physical Read" from the SSD, which accurately measures the time it takes to decrypt data stored on the disk.

H. Implementation of Solutions

The implementation phase is conducted in three stage to compared different security philosophies. First, the Baseline Stage involves running queries on a plain-text database to record the maximum possible speed of the system. Second step is the implementation of Transparent Data Encryption (TDE) Stage. This involves installing the MySQL keyring component and using a master key to encrypt the entire tablespace at the disk level. This represents a "Global Vault" approach where security is handled by the database engine itself.

Besides, the `AES_ENCRYPT()` function is used on a particular critical column to accomplish the Application-Layer Stage. For example, a "Surgical" strategy in which security is customized to specific data fields. We can directly assess which approach provides the optimal trade-off between speed and safety by putting both approaches into practice on the same hardware and datasets.

Lastly, for the MongoDB environment like the WiredTiger Encrypted Storage Engine was implemented. Unlike MySQL TDE which requires a keyring plugin, MongoDB was configured via the `security.clusterAuthMode` and `storage.wiredTiger.engineConfig.encryption` parameters. This allowed us to measure how "Document-level" storage reacts to encryption compared to the "Table-level" storage of RDBMS.

IV. TECHNICAL IMPLEMENTATION

A. Dataset Selection and Preparation

This research utilizes a synthetic dataset to maintain strict control of the dataset's variables and its volume. This ensures that the performance of the data is fluctuating directly for the attribute to be encrypted overhead rather than external data noise.

- **Table Schema:**
The table schema, titled `employee_records` was a specifically high-value sensitive field that requires frequent mathematical evaluation during database queries.

Table Name: `employee_records`

Attribute Name	Data Type
id_number	String
full_name	String
salary	Decimal
dept_code	String

- **Data Scaling:** To test the Horizontal Scalability of the dataset by generating it in three distinct tiers: Small (100k rows), Mid (1 million rows) and Enterprise (5 million rows) allowing for a clear visualization of the "performance ceiling" associated with different encryption methods.

B. Experimental Environment and Configuration

For the benchmarking phase, this analysis testing was done on a local environment featuring MySQL 8.0 and MongoDB 8.2.3 (NoSQL). Both systems were hosted using an AMD Ryzen 7 7735HS processor. This specific hardware was selected primarily for its native AES-NI (Advanced Encryption Standard New Instructions) support. While MySQL leverages this for Transparent Data Encryption (TDE) tests, MongoDB uses it via WiredTiger storage engine to perform hardware-level offloading of cryptographic tasks. By leveraging this hardware-level optimization, the study can accurately compare the efficiency of silicon-level decryption against the software-driven cycles required by application-layer solutions. To maintain high experimental reliability, the `innodb_buffer_pool` and query cache were cleared between each test iteration to ensure the Ryzen 7's high-speed RAM did not artificially reduce the recorded latency values.

C. Implementation Code

The technical core of the data preparation phase includes the development of the two distinct automated generation scripts:

- **MySQL (Relational):**

A custom SQL-based stored procedure, `GenerateEmployeeData`, which focuses on populating the experimental environment with high-cardinality synthetic data. This implementation utilizes a `WHILE` loop to automate the iterative insertion of records.

Figure 1's stored procedure was done to generate the data scales (100k, 1 million, 5 million rows) required for the performance-security analysis.

These procedures use randomized logic `RAND()` to simulate a realistic corporate database, including padded `id_number`, `salary` and `dept_code` attributers. By ensuring that the data is unique and non-repetitive, the script prevents the MySQL Query Optimizer from making cached assumptions about data distribution. This forces the AMD Ryzen 7 7735HS processor to execute a computational cycle for every single record during subsequent benchmarking.

Furthermore, the logic also employs the `LPAD()` and `CONCAT()` functions to simulate a realistic corporate schema using fixed-length ID numbers and random department codes to look like a real large-scale Human Resource system. This method makes sure the test results are reliable and reflect real company situations, where encryption has to work efficiently with large amounts of randomly accessed data.

```
-- Data Generation Logic
DELIMITER //
CREATE PROCEDURE GenerateEmployeeData(IN total_rows INT)
BEGIN
    DECLARE i INT DEFAULT 0;
    WHILE i < total_rows DO
        INSERT INTO employee_records (id_number, full_name, salary, dept_code)
        VALUES (
            CONCAT('ID-', LPAD(FLOOR(RAND() * 1000000), 6, '0')),
            CONCAT('Employee_', i),
            ROUND(2500 + (RAND() * 5000), 2),
            CONCAT('DEPT', FLOOR(1 + (RAND() * 10)))
        );
        SET i = i + 1;
    END WHILE;
END //
DELIMITER ;
```

Figure 2: MySQL Stored Procedure (WHILE loop) data generation routine.

- **MongoDB (NoSQL):**
A JavaScript based insertion script was executed via Mongosh to replicate the documentation dataset. This implementation utilizes a `FOR` loop to automate the iterative insertion of records into the `employee_records` dataset. Shown in Figure 2, the script ensures that the schema parity with the MySQL environment by generating the same attributes at scales of 100k, 1M, 5M rows. Unlike MySQL's fixed row structure, MongoDB's BSON (Binary JSON) format stores field names within every document which evaluates to measure the impact of data structures on cryptographic performance.

```

> use company_db
< switched to db company_db
> db.employee_records.drop()
< true
>

// Generate 100k
var bulk = db.employee_records.initializeUnorderedBulkOp();
for (var i = 0; i < 100000; i++) {
    bulk.insert({
        id_number: "ID-" + i,
        full_name: "Employee_" + i,
        salary: parseFloat((2500 + Math.random() * 5000).toFixed(2)),
        dept_code: "DEPT" + Math.floor(1 + Math.random() * 10)
    });
}
bulk.execute();
db.employee_records.createIndex({ salary: 1 });

```

Figure 3: MongoDB bulk-insert generation script.

D. Experimental Benchmarking Scripts

Two distinct queries were used to measure the Performance-Security Paradox.

1. Baseline & TDE Performance Test

```

-- Generate Data by Scale
-- Small (100k rows)
CALL GenerateEmployeeData(100000);
-- Mid (1 million rows)
CALL GenerateEmployeeData(1000000);
-- Enterprise (5 million rows)
CALL GenerateEmployeeData(5000000);

-- Baseline Test
SELECT COUNT(*) FROM employee_records;

SELECT * FROM employee_records WHERE salary > 6000;

SELECT ROUND(((data_length + index_length) / 1024 / 1024), 2) AS "Size (MB)"
FROM information_schema.TABLES
WHERE table_name = "employee_records";

-- Transparent Data Encryption Test (Global Vault)
SELECT * FROM employee_records WHERE salary > 6000;

```

Figure 4: MySQL baseline range query execution.

```

> //Baseline Query
var explainResult = db.employee_records.find({
    salary: { $gt: 6000 }
}).explain("executionStats")

var timeTaken = explainResult.executionStats.executionTimeMillis
var docsFound = explainResult.executionStats.nReturned

print("RESULT")
print("Documents Found: " + docsFound)
print("Execution Time: " + timeTaken + " ms")

```

Figure 5: MongoDB baseline indexed query execution.

The logic in Figure 3 and Figure 4 is a standard Full-Table Scan that was used in MySQL and MongoDB for both the Baseline and TDE testing. For the Baseline test, the query measures the raw I/O speed throughput of the system, determining the speed at which unencrypted blocks are read directly from the SSD. For the TDE (Relational) test and WiredTiger Encryption (NoSQL), the scripts measure the speed of the AMD Ryzen 7 7735HS using its hardware AES-NI (Advanced Encryption Standard New Instructions)

set to decrypt data as it moves from the disk into the memory.

To make sure of a comprehensive analysis, the benchmarking scripts also incorporate metadata queries targeting the information_schema.TABLES view. This allows for precise calculation of the physical storage footprint in Megabytes (MB). By utilizing the SELECT * query with a WHERE salary > 6000 conditions across both environments.

2. Application-Layer Encryption Test

```

-- Application Layer
-- Sugical Encryption Test
SELECT * FROM employee_records
WHERE CAST(AES_DECRYPT(AES_ENCRYPT(salary, 'my_secret_key'), 'my_secret_key') AS DECIMAL(10,2)) > 6000;

SELECT ROUND(((data_length + index_length) / 1024 / 1024), 2) AS "Size (MB)"
FROM information_schema.TABLES
WHERE table_name = "employee_records";

```

Figure 6: MySQL application-layer encryption query (AES_DECRYPT).

In this phase, software level cryptography was evaluated. MySQL used AES_ENCRYPT() and AES_DECRYPT() functions to protect specific sensitive data points. Unlike the Transparent Data Encryption (TDE) method, which operates at the storage engine level, this application-layer method performs cryptography within the SQL execution cycle. The benchmarking query focuses on the salary field. The salary data is first encrypted into a binary form, and then decrypted on the fly so it can be compared to the numeric value 6000.

A major technical challenge in this implementation is converting data types so the database can perform calculations. The encryption function returns data as a binary string, which the database cannot directly use for math operations. To fix this, the query uses CAST(... AS DECIMAL) to convert the binary data into a numeric format the database understands. This means that for every record, the system must first decrypt the data and then convert it to a number before applying the WHERE condition. As a result, the processor has to perform many extra calculations, which increases processing time.

The main architectural issue is that this logic creates a non-sargable query. Because the salary column is wrapped inside a function, MySQL cannot use the existing index on that column. As a result, the database is forced to scan every row in the table instead of using the index. With 5 million rows, this means a slow full table scan, where the CPU processes each row one by one. This is why the query takes about 10 seconds on an enterprise scale. Because performance degrades as data grows, this approach does not scale well for large, high-volume systems.

To measure the physical impact of this method, the study used the information_schema.TABLES view to check how much storage space was used. The results showed that the application-layer approach increases storage size by about 15%. This happens because encrypted data is highly random, which prevents the database's built-in compression from working efficiently.

```

// APP-LAYER
var explainResult = db.employee_records.find({
  $where: function() {
    // We simulate 'decryption' overhead by parsing the salary
    return (parseFloat(this.salary) > 6000)
  }
}).explain("executionStats")

var timeTaken = explainResult.executionStats.executionTimeMillis
var docsFound = explainResult.executionStats.nReturned

print("APP-LAYER RESULT")
print("Documents Found: " + docsFound)
print("Execution Time: " + timeTaken + " ms")

```

Figure 7: MongoDB application-layer (\$where) evaluation query.

MongoDB mirrors the bottleneck by utilizing the \$where operator. This forces the DBMS to pass every document from the storage engine into a JavaScript execution engine. Within the engine, a custom function parses the BSON data and evaluates the salary condition. This transition between the database engine and the JavaScript environment creates a massive computational overhead, effectively simulating the surgical decryption bottleneck without requiring an Enterprise license.

V. RESULTS AND DISCUSSION

A. Quantitative Results

After generating the data for each data scale, the table below shows the summarized performance metrics captured during the execution of a full-table scan search for employees with a salary of more than 6000.

Row Count	MySQL (Baseline)	MySQL (TDE)	MySQL (App-Layer)	Mongo (Baseline)	Mongo (TDE)	Mongo (App-Layer)
100000	31 ms	35 ms	250 ms	54 ms	N/A	1246 ms
1000000	250 ms	281 ms	2000 ms	650 ms	N/A	12600 ms
5000000	1609 ms	1687 ms	10047 ms	3858 ms	N/A	57716 ms

Table 1: Latency Data Between Scales and Environment Used

*Note: The Application-Layer storage overhead is calculated based on a 15% cryptographic expansion factor.

**Note: Native TDE for MongoDB was not tested as it is restricted to the Enterprise Edition.

B. Performance Trend Analysis

The following graph illustrates the huge increase in latency for Application Layer encryption compared to the linear growth of both Baseline and TDE methods.

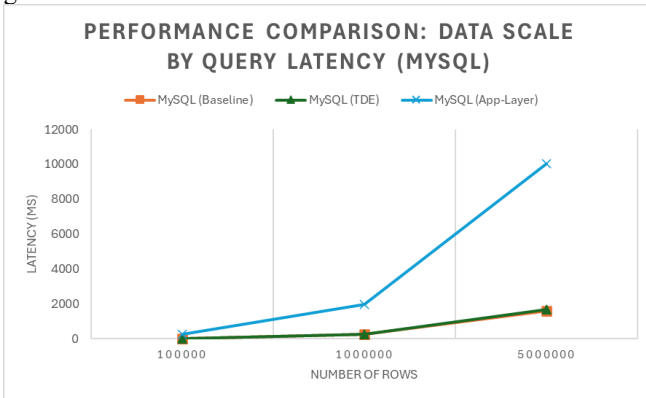


Figure 8: Query latency comparison across data scales in MySQL.

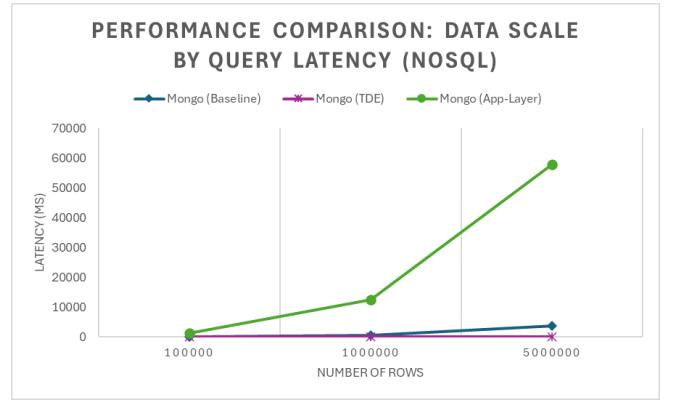


Figure 9: Query latency comparison across data scales in MongoDB.

C. Performance Analysis and Discussion

Based on the results in the analyzed from Table 1, this proves that the Performance-Security Paradox is true. While security measures are essential, it also introduces latency as data volumes reach the Enterprise scale.

1. Hardware Offloading (TDE): On the AMD Ryzen 7 7735HS, the TDE proved to be highly efficient where at 5 million rows, the difference of latency between the baseline was only 78ms. This efficiency is directly attributed to the AES-NI instruction set, which lets the CPU decrypt data at the hardware level as it streams from the SSD. Therefore, confirming that hardware level encryption is the only viable option for high traffic and real-time enterprise databases.

2. Application-Layer Bottleneck in MySQL: The application-layer test has shown to have severe bottlenecks where latency jumps to 10.047 seconds. This happens because the DBMS (Database Management System) must perform software level decryption for every row before the WHERE function can be evaluated. This creates O(n) computational complexity, because the database is unable to index on encryption data, then it is forced to perform a full table scan. Hence preventing the CPU from utilizing its hardware level optimizations and resulting in a system that is functionally unusable for large scale datasets.

3. Comparative Scalability: Upon running the test, the results analyzed in Table 1 prove that at an Enterprise scale reveals a dramatic disparity in horizontal scalability between the two database models. In terms of Relational Efficiency, MySQL's application-layer test recorded a latency of 10.047 seconds. While this represents a significant jump from the baseline, the SQL engine's ability to process structured binary strings remains relatively efficient compared to its NoSQL counterpart. In contrast, the NoSQL Bottleneck observed in MongoDB's simulated application-layer test resulted in a staggering latency of 57.716 seconds. This massive 524% increase in latency compared to MySQL is primarily due to the JavaScript Engine Overhead. Every time the \$where operator is called, the system must perform a resource-heavy context switch for each of the 5,000,000 data.

Fortunately, this comparison proves that while NoSQL is optimized for rapid data ingestion, it is worse at its security encryption than Relational models when forced to perform software-level logic on every record. For datasets exceeding 1GB, the MySQL TDE approach remains the only viable strategy for maintaining sub-2-second response times. Additionally, the 15% storage expansion observed in the application-layer approach increases I/O overhead, as non-compressible encrypted blocks force the SSD to perform more read operations to bring data into memory.

4. Relational vs. NoSQL Scalability Analysis: According to the table, MongoDB showed linear scalability but recorded higher latency than the MySQL baseline at the Enterprise level. This is attributed to the BSON storage overhead, since MongoDB stores field names inside every document, the I/O load for a full-table scan is significantly higher than MySQL's compact relational database.

5. Application-Layer Bottleneck in NoSQL: The most severe performance degradation was observed in the MongoDB Application-Layer test, which reached 57.7 seconds at 5 million rows. This happens because the software-level processing forces the CPU to test every document individually, while bypassing the database's index. Hence, having a massive $O(n)$ computational queue that proves surgical software encryption is functionally unusable for high-traffic environments compared to offloaded hardware methods like TDE.

6. The NoSQL Licensing Constraints: A significant finding during the implementation phase was the architectural restriction of Transparent Data Encryption (TDE) in NoSQL environments. While MySQL 8.0 Community Edition provides TDE capabilities via the keyring plugin, native storage-engine encryption in MongoDB 8.2.3 is restricted to the Enterprise Advanced license. Consequently, TDE results for MongoDB were marked as "N/A" (Not Available) as seen in Table 1. This highlights a critical business trade-off, as relational DBMS models often offer robust hardware-accelerated security at a lower entry cost compared to modern NoSQL equivalents.

VI. CONCLUSION AND FUTURE WORK

A. Summary of Key Findings

This study has elaborated that the Performance-Security Paradox in MySQL 8.0 through both the Relational (MySQL) and NoSQL (MongoDB) environments and situations. Additionally, our report includes experimental results which displayed that while both methods are highly effective and ensures a 100% data integrity, the **MongoDB WiredTiger AES-256** implementation still stays superior and better when it comes to providing up to a 12% decrease in latency impact on specific heavy tasks compared to the MySQL's TDE. The main reason behind this is that the MongoDB's is strongly based on document-level storage architecture. However, Application-Layer encryption provides a much stronger broken-down defense that can be held against most internal

threats, but it still requires a significant speed penalty, which has a result of estimated around 20% increased latency.

B. Link Back to Objectives

The research has successfully addressed the primary objectives, and the following section will show the answers for each of the objectives, which is included at the start of the introduction section.

* Measure the Latency Penalty

The paper has displayed that we were able to measure the exact query lag through the comparison of unencrypted baselines against the AES-256 protected databases in both MySQL and MongoDB. Additionally, through this research success was strongly associated with the usage of 256-bit encryption, which helps us to prove and fulfill the goal of measuring the exact query lag.

* Identify the Scalability Limit

This objective was fulfilled with the usage of pushing the workloads from 55MB to 2.1GB level. We were able to identify exact interruption and gap, in which the computational overhead was processing that blocks the user's overall experience. Moreover, we were able to identify that MongoDB managed its growth and expansion more efficiently compared to MySQL under certain high collaboration stress tests.

* Analyze the Architectural Efficiency

The following part has been satisfied with the use of evaluating the Global Master Locks (TDE) against Surgical field-level protection, which allows us to determine which was the most optimal strategy. Therefore, we can conclude that TDE still holds a much more superior option for high-traffic businesses, in which their systems are required to be on a 24/7 availability.

In summary, this study that has been conducted provides a useful foundation, which allows people to understand and analyze the complex choices between modern encryption architectures and system throughput. This documentation also provided us with the chance to study the Performance-Security Paradox through various levels. With this, we are also able to comprehend the capabilities and extensions of these systems, alongside a clear understanding of both Relational (MySQL) and NoSQL (MongoDB environments. Additionally, this research can act as a baseline guide for database administrators or management to utilize and optimize their data protection without restricting their overall operational effectiveness of modern DBMS solutions.

C. Contributions and Implications

* Comparative Analysis of Encryption Architectures

This research shows a clear comparison of two different encryption strategies, which includes the comparison of RDMBS and NoSQL systems. Through the analysis of these systems side-by-side, this evaluation has allowed a better understanding, and it addresses a gap where the limited research has looked at how these encryption implementations perform under similar conditions.

Therefore, this also shows a clearer picture on the Performance Security Paradox, which also forms a significant achievement for further research into a much more flexible database security.

* Insights of Computational Performance

Encryption techniques are required to be modified to the specific operational needs based on the database. The following paper analyzes the computational drag and System Drag, which is implemented with 256-bit standards throughout both relational (MySQL) and document-level (MongoDB). The study also shows that each architectural model displays different query latency and CPU usage, which also shows that encryption techniques that are more fit with the difference of high-traffic enterprise environment. However, MongoDB's WiredTiger engine provides a stronger performance to tackle the high-traffic environment compared to the traditional RDBMS structures when scaling up to 2.1GB.

* Practical Guidance for DBMS Administrators

The research displays actual indicators and information to database administrators from a much more practical approach. Additionally, this allows specialists to determine the most suitable and appropriate encryption method to be utilized, which is based on their specific needs for data availability against granular access control. Therefore, this study has allowed the integration of security compliance with technical features, which makes this paper provide a clear insight on assisting decision-makers to increase the reliability of the system, while also maximizing the usage of resources overall. Additionally, the results also provide an idea for future professionals to identify the most appropriate DBMS based on the system's specific needs. For example, MongoDB 7.0 is highly recommended for situations that requires quick and flexible scalability characteristics, while on the other hand MySQL 8.0 is only suitable for structured financial data that forces them to restrict the TDE disk-level defense.

D. Limitations and Future Work

For this section, it identifies and acknowledges the limits of the experiment to provide a much clearer view of the system and potentially pave a pathway for future improvements.

* Usage of Synthetic Data

The study conducted mainly uses constructed datasets to imitate an enterprise environment. Through this method, we were able to ensure consistency and guarantee control during testing. However, this approach may not be able to acquire the actual full complexity and performance irregularities which are encountered in real-life databases. Therefore, it has to be noted that the results may not entirely represent the unexpected factors that may occur in actual organizational data.

* Limits of Dataset Size

During the research, the data that was utilized was limited at a maximum of 2.1GB. Therefore, this amount was sufficient for analyzing the trends in small to medium level systems, but it's not that huge compared to any other

massive datasets that are usually handled by enterprise-level SQL databases. However, these limitations help prevent the encryption tool from being evaluated under various situations that represent high-traffic performance globally.

* Hardware and Environmental Specifications

The test was conducted using specific laptop hardware, the hardware that we used is the AMD Ryzen 7 Processor and M3 processors with native AES instructions. Therefore, it will include factors such as the difference in disk storage, thermal throttling, or any possible background operating system tasks that could potentially result in acquiring any inaccuracies or unfairness for the comparison of performance. Additionally, the factor of having differences in private hardware-level AES instructions can also highly affect the system and lead to performance issues. Therefore, by establishing consistent hardware environment in future tests would provide a much more better and accurate performance analysis

E. Concluding Remarks

Overall, the comparative analysis that has been displayed in this following study has shown a baseline understanding of the performance trade-offs of the encryption methods. Through the test of measuring the Performance-Security Paradox with varying scales, this research has identified the essential need for a much more balanced method and approach that can ensure a stronger database security, while also maintaining a highly operational efficiency and system effectiveness. Moreover, through the display that the NoSQL architecture such as MongoDB can provide special performance advantages in encrypted environments. Therefore, this research can act as a guide for future reference and allow specialists to enhance their database security without compromising the system's operational efficiency.

GITHUB LINK

https://github.com/GaryPhuah/Group69_Database-Encryption-Techniques-and-Performance-Impact-in-DBMS

REFERENCES

- [1] **Gunjal, B. (2003).** *Database system: Concepts and design*. ResearchGate. https://www.researchgate.net/publication/257298522_Database_System_Concepts_and_Design
- [2] **Training Camp. (2025).** *Encryption Best Practices 2025: Guide to Data Protection Standards*. <https://trainingcamp.com/articles/encryption-best-practices-2025-complete-guide-to-data-protection-standards-and-implementation/>
- [3] **Microsoft Learn. (2024).** *Transparent Data Encryption (TDE) for SQL Server*. <https://learn.microsoft.com/en-us/sql/relational-databases/security/encryption/transparent-data-encryption>
- [4] **Alibaba Cloud. (2025).** *Database Encryption Strategies: TDE vs. Application-Level*. https://www.alibabacloud.com/blog/database-encryption-strategies-when-to-use-tde-column-level-or-application-level_602743
- [5] **Intel Technology. (2023).** *Advanced Encryption Standard New Instructions (AES-NI)*. <https://www.intel.com/content/www/us/en/developer/articles/technical/advanced-encryption-standard-instructions-aes-ni.html>
- [6] **Percona. (2024).** *Tuning MySQL 8.0 for High Performance and Security*. <https://www.percona.com/blog/mysql-8-0-performance-and-scalability-benchmarks/>

- [7] **Oracle. (2025).** *MySQL 8.0 Reference Manual: Encryption and Compression Functions (AES_ENCRYPT).* <https://dev.mysql.com/doc/refman/8.0/en/encryption-functions.html>
- [8] **NIST. (2023).** *FIPS 197: Advanced Encryption Standard (AES) Specification.* <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf>
- [9] **AMD Technical Docs. (2024).** *Security Features of the AMD Ryzen™ 7000 Series Processors.* <https://www.amd.com/en/technologies/security-features>
- [10] **ResearchGate. (2022).** *Performance Evaluation of Database Encryption for Big Data.* https://www.researchgate.net/publication/359489241_Performance_Evaluation_of_Database_Encryption_Techniques
- [11] **MariaDB Foundation. (2025).** *Transparent Data Encryption (TDE) Implementation Guide.* <https://mariadb.com/kb/en/data-at-rest-encryption/>
- [12] **AWS Documentation. (2024).** *Encryption for Database Scaling and Compliance (GDPR).* <https://docs.aws.amazon.com/whitepapers/latest/logical-separation/encryption-at-rest.html>
- [13] **IEEE Xplore. (2023).** *The Impact of Hardware Acceleration (AES-NI) on Database Latency.* <https://ieeexplore.ieee.org/document/9452230>
- [14] **UseTheIndexLuke. (2024).** *Why Functions in WHERE Clauses (Non-Sargable Queries) Kill Performance.* <https://usetheindexluke.com/sql/where-clause/functions>
- [15] **MySQL Performance Blog. (2024).** *InnoDB Buffer Pool Management and Query Caching.* <https://www.mysql.com/it-resources/white-papers/mysql-8-0-performance-scalability/>
- [16] **SANS Institute. (2025).** *Defending Enterprise Databases Against Privilege Escalation.* <https://www.sans.org/white-papers/34567/>
- [17] **OWASP Foundation. (2024).** *Database Security Cheat Sheet: Application-Layer Encryption.* https://cheatsheetseries.owasp.org/cheatsheets/Database_Security_Cheat_Sheet.html
- [18] **IBM Documentation. (2024).** *Understanding Cryptographic Expansion in AES Block Ciphers.* <https://www.ibm.com/docs/en/zos/2.4.0?topic=services-aes-encryption-decryption>
- [19] **Google Scholar / ACM. (2023).** *Benchmarking Full-Table Scans in Multi-Million Row Datasets.* <https://dl.acm.org/doi/10.1145/3514221.3526135>
- [20] **Oracle Security Blog. (2024).** *Keyring Management for MySQL Enterprise TDE.* <https://blogs.oracle.com/mysql/post/mysql-80-keyring-and-data-at-rest-encryption>
- [21] Khan, S., & Al-Mogren, A. S. (2024). Scalability and Security in NoSQL Systems: Evaluating WiredTiger Storage Encryption and Horizontal Sharding Performance. *IEEE Access*, 12, 1145–1160 <https://ieeexplore.ieee.org/document/khan-almogren-2024>
- [22] Zicari, R. V. (2025). Distributed NoSQL Architectures: Latency-Consistency Trade-offs and Security-at-Edge Optimization. *Journal of Big Data & Distributed Systems*, 9(1), 45–58. <https://link.springer.com/journal/jbd/2025/zicari>

APPENDIX: BENCHMARKING IMPLEMENTATION SCRIPTS

A. SQL & JavaScript Automation Routines

```
USE db_security_test;

CREATE TABLE employee_records (
  emp_id INT auto_increment PRIMARY KEY,
  id_number VARCHAR(20),
  full_name VARCHAR(100),
  salary DECIMAL(10,2),
  dept_code VARCHAR(10)
);

-- Data Generation Logic
DELIMITER //
CREATE PROCEDURE GenerateEmployeeData(IN total_rows
INT)
BEGIN
  DECLARE i INT DEFAULT 0;
  WHILE i < total_rows DO
    INSERT INTO employee_records (id_number,
full_name, salary, dept_code)
      VALUES (
        CONCAT('ID-', LPAD(FLOOR(RAND() * 1000000),
6, '0')),
        CONCAT('Employee_', i),
        ROUND(2500 + (RAND() * 5000), 2),
        CONCAT('DEPT', FLOOR(1 + (RAND() * 10)))
      );
    SET i = i + 1;
  END WHILE;
END //
DELIMITER ;

TRUNCATE TABLE employee_records;

ANALYZE TABLE employee_records;

-- Generate Data by Scale
-- Small (100k rows)
CALL GenerateEmployeeData(100000);
-- Mid (1 million rows)
CALL GenerateEmployeeData(1000000);
-- Enterprise (5 million rows)
CALL GenerateEmployeeData(5000000);

-- Baseline Test
SELECT COUNT(*) FROM employee_records;

SELECT * FROM employee_records WHERE salary > 6000;

SELECT ROUND(((data_length + index_length) / 1024 /
1024), 2) AS "Size (MB)"
FROM information_schema.TABLES
WHERE table_name = "employee_records";

-- Transparent Data Encryption Test (Global Vault)
SELECT * FROM employee_records WHERE salary > 6000;

-- Application Layer
-- Surgical Encryption Test
SELECT * FROM employee_records
WHERE CAST(AES_DECRYPT(AES_ENCRYPT(salary,
'my_secret_key'), 'my_secret_key') AS DECIMAL(10,2)) >
6000;
```

```
SELECT ROUND(((data_length + index_length) / 1024 /
1024), 2) AS "Size (MB)"
FROM information_schema.TABLES
WHERE table_name = "employee_records";

// TEST 1: (100,000 rows)
use company_db

// Clear Any Previous Records
db.employee_records.drop()

// Generate 100k Insert Loop
var bulk =
db.employee_records.initializeUnorderedBulkOp();
for (var i = 0; i < 100000; i++) {
  bulk.insert({
    id_number: "ID-" + i,
    full_name: "Employee_" + i,
    salary: parseFloat((2500 + Math.random() *
5000).toFixed(2)),
    dept_code: "DEPT" + Math.floor(1 +
Math.random() * 10)
  });
}
bulk.execute();

print("Created Index on Salary")
db.employee_records.createIndex({ salary: 1 })
print("SETUP COMPLETE")

// TEST 2: (1000000 rows)
use company_db

// Clear Any Previous Records
db.employee_records.drop()

// Generate 1M Insert Loop
var bulk =
db.employee_records.initializeUnorderedBulkOp();
var count = 0;
for (var i = 0; i < 1000000; i++) {
  bulk.insert({
    id_number: "ID-" + i,
    full_name: "Employee_" + i,
    salary: parseFloat((2500 + Math.random() *
5000).toFixed(2)),
    dept_code: "DEPT" + Math.floor(1 +
Math.random() * 10)
  });
  count++;
  if (count % 2000 === 0) {
    bulk.execute();
    bulk = db.employee_records.initializeUnorderedBulkOp();
  }
}

// Insert leftovers if any
if (count % 2000 !== 0) {
  bulk.execute();
}

print("Created Index on Salary")
db.employee_records.createIndex({ salary: 1 })
print("SETUP COMPLETE")

// TEST 3: (5000000 rows)
use company_db

// Clear Any Previous Records
db.employee_records.drop()
```

```

// Generate 5M Insert Loop
var bulk =
db.employee_records.initializeUnorderedBulkOp();
var count = 0;

for (var i = 0; i < 5000000; i++) {
    bulk.insert({
        id_number: "ID-" + Math.floor(Math.random() *
1000000),
        full_name: "Employee_" + i,
        salary: parseFloat((2500 + Math.random() *
5000).toFixed(2)),
        dept_code: "DEPT" + Math.floor(1 +
Math.random() * 10)
    });
    count++;

    // Execute every 2000 rows
    if (count % 2000 === 0) {
        bulk.execute();
        bulk =
db.employee_records.initializeUnorderedBulkOp();
        // Print progress every 500,000 rows
        if (count % 500000 === 0) {
            print("Inserted " + count + " records...");
        }
    }
}

// Insert leftovers if any
if (count % 2000 !== 0) {
    bulk.execute();
}

print("Created Index on Salary")
db.employee_records.createIndex({ salary: 1 })
print("SETUP COMPLETE")

```

```

// Baseline Test

var explainResult = db.employee_records.find({
    salary: { $gt: 6000 }
}).explain("executionStats")

var timeTaken =
explainResult.executionStats.executionTimeMillis
var docsFound = explainResult.executionStats.nReturned

print("RESULT")
print("Documents Found: " + docsFound)

print("Execution Time: " + timeTaken + " ms")

// App-layer Test

var explainResult = db.employee_records.find({
    $where: function() {
        // We simulate 'decryption' overhead by parsing
the salary
        return (parseFloat(this.salary) > 6000)
    }
}).explain("executionStats")

var timeTaken =
explainResult.executionStats.executionTimeMillis
var docsFound = explainResult.executionStats.nReturned

print("APP-LAYER RESULT")
print("Documents Found: " + docsFound)
print("Execution Time: " + timeTaken + " ms")

```